

Full Stack / Web Design

1)Ecommerce

Project Name: Tetravate

Purpose: Tetravate is an e-commerce website designed to enable users to browse, explore, and purchase products online through a user-friendly interface.

Problem Statement: Traditional shopping can require customers to visit multiple stores to compare products and make purchases. Tetravate provides a centralized online platform for product discovery, shopping, and order management.

Features: The website includes product browsing, category-wise navigation, search and filtering, product details, shopping cart, wishlist, user registration and login, role-based customer and admin access, profile management, delivery address management, product reviews, order placement, order history, order tracking, and administrative store management.

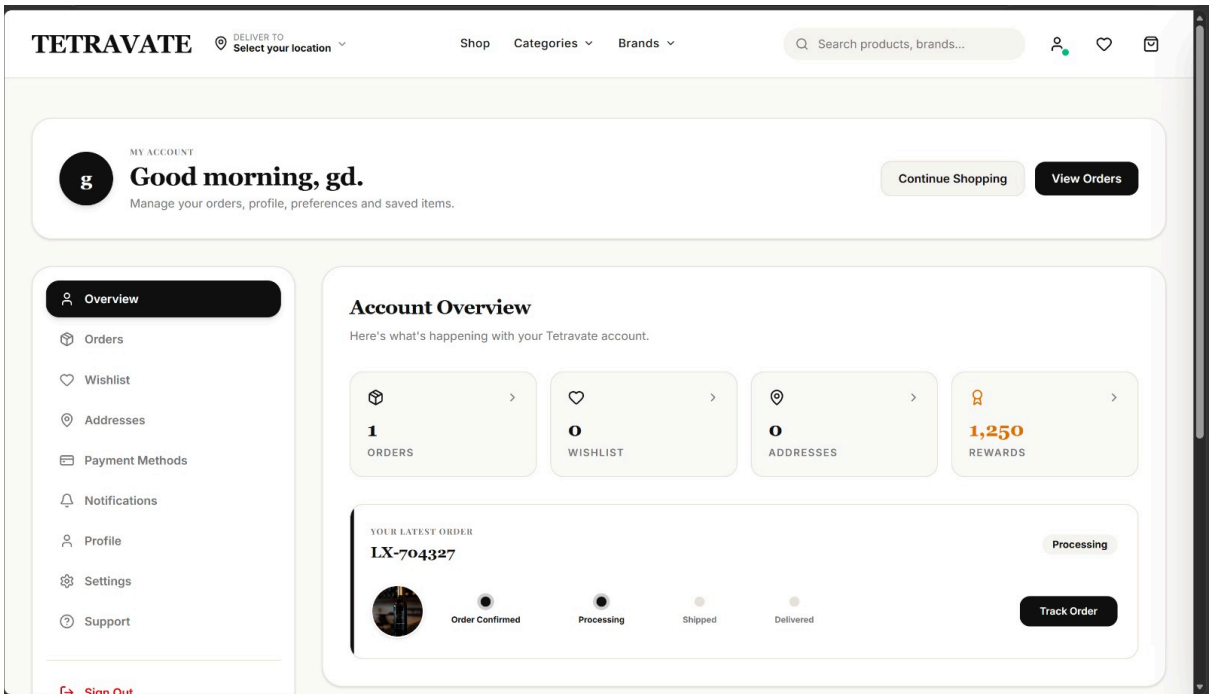
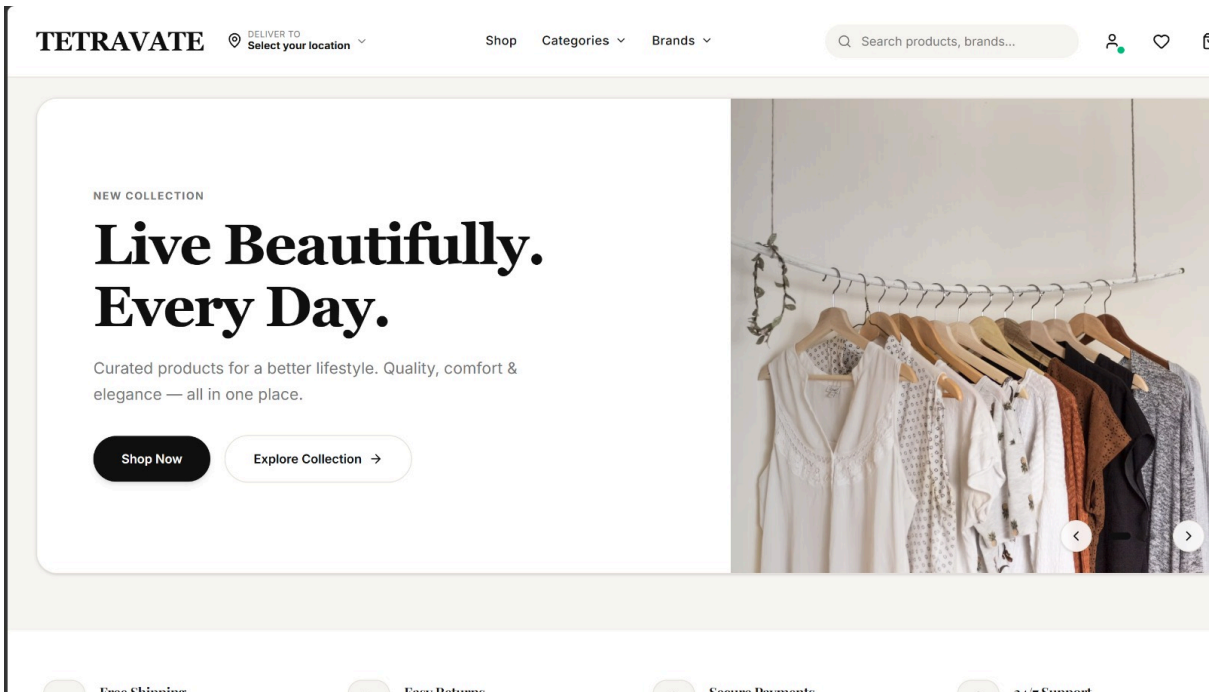
Tech Stack: The project uses a web development technology stack consisting of frontend technologies, backend services, database integration, authentication mechanisms, and development tools. The exact languages, frameworks, and database should be confirmed from the project source code.

Category: E-commerce and Full-Stack Web Development.

Project Status: Under development. The final status should be updated to deployed application, live website, or prototype based on the current implementation.

Contributions: The project involves developing the website interface, product and category pages, authentication, customer and administrator role management, cart and wishlist functionality, profile and address management, product reviews, order handling,

and administrative features. The contribution details should reflect the modules actually implemented by the project team.



2)RESTAURANT

Project 1: MĀRA — Modern Indian Dining

Live Demo: <https://mara-restaurant-28hy.vercel.app/>

1. Project Name

MĀRA — Modern Indian Dining

2. Purpose

To create a premium, interactive restaurant website that allows customers to explore the restaurant, discover menu items, view food photography, and access online ordering and table reservation interfaces.

3. Problem

Many restaurant websites provide limited menu information, inconsistent food imagery, and complicated navigation. Customers need an attractive, easy-to-use platform where they can explore dishes, understand the dining experience, and access ordering and reservation options from one place.

4. Main Features

Premium restaurant homepage with a strong visual identity.

Interactive food menu with categories and dish details.

Food photography and restaurant gallery.

Restaurant story and dining experience information.

Private dining information.

Online ordering interface with an order tray/cart.

Table reservation interface.

Customer, staff, and admin access interfaces, where implemented.

Responsive layouts for desktop and mobile devices.

Animations, interactive elements, and navigation.

Live deployment using Vercel.

5. Screenshots and What Each Shows

Homepage: Restaurant branding, navigation, hero section, and primary actions.

Menu Page: Food categories, dish listings, images, and prices.

Dish Details Page: Selected dish image, description, price, and ordering options.

Gallery Page: Restaurant atmosphere, food photography, and visual presentation.

Reservation Page: Interface for entering table reservation details.

Order Tray / Cart: Selected items and the order review interface.

Our Story Page: Restaurant concept, dining philosophy, and brand identity.

Private Dining Page: Information about private dining and group experiences.

Contact Page: Restaurant contact and location information.

Login / Access Screen: Customer, staff, and admin entry interfaces, if included in the final implementation.

6. Tech Stack

Languages: TypeScript, JavaScript, HTML, CSS

Frontend: React

Build Tool: Vite

Styling: Tailwind CSS and/or project CSS

Deployment: Vercel

Version Control: Git and GitHub

7. Category

Web Development · Restaurant & Hospitality Technology · UI/UX Design

8. Project Status

Live deployed demo / Portfolio project

The website is deployed on Vercel and can be accessed publicly. It is a portfolio concept and should not be presented as a real restaurant client project.

9. Contributions

Designed and developed the restaurant website interface.

Implemented restaurant navigation and individual content pages.

Built menu browsing and dish-detail interfaces.

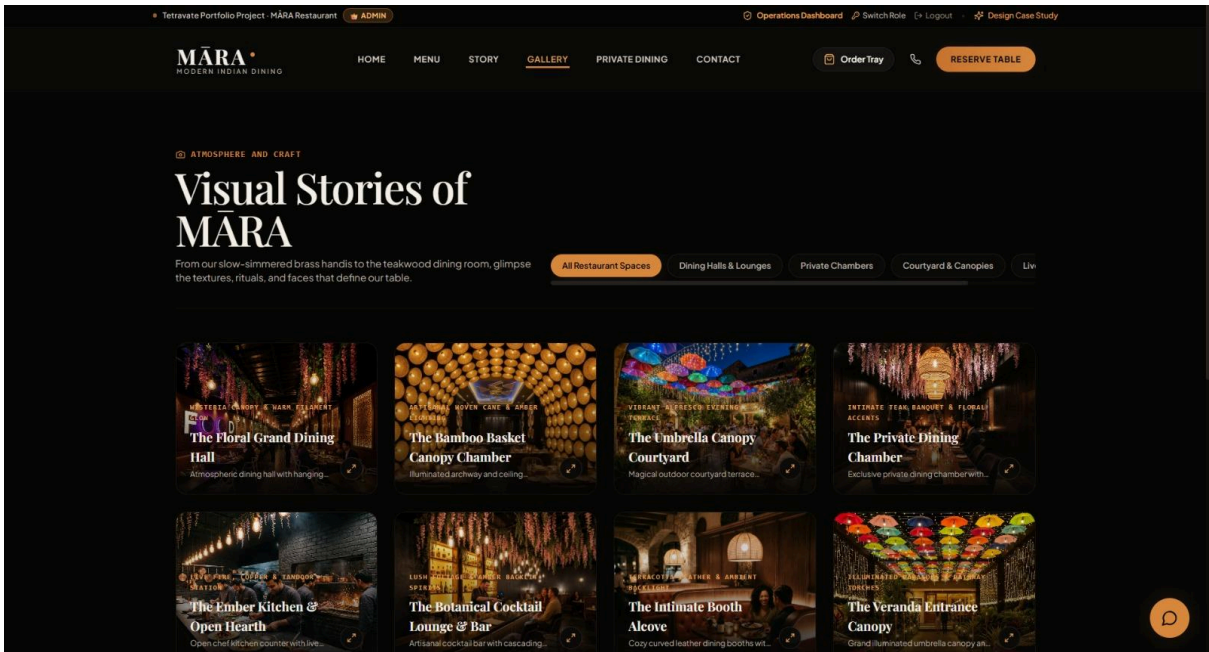
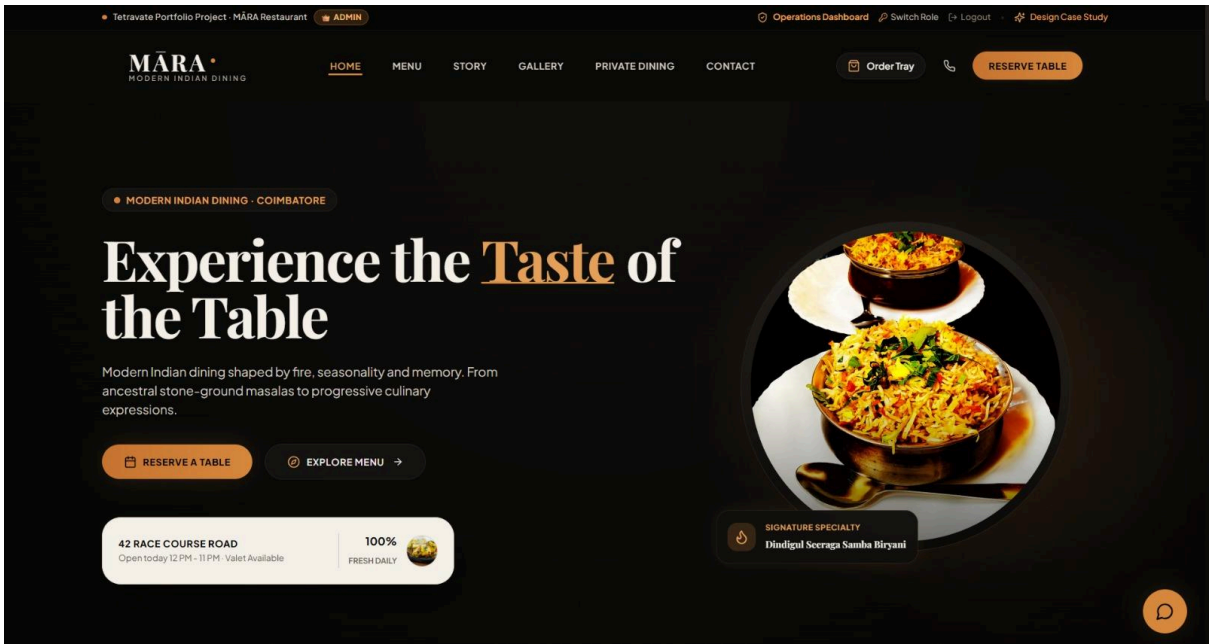
Developed ordering and reservation user interfaces.

Created responsive layouts and interactive visual elements.

Organized restaurant content and menu presentation.

Connected the project to GitHub and deployed it using Vercel.

Tested the deployed website and reviewed the user experience.



3)COLLEGE MANAGEMENT

Project Details — Nivara ERP

1. Project Name

Nivara ERP — College Management System

2. Purpose

Nivara ERP is a role-based college management system designed to centralize academic administration, student services, examinations, financial operations, and placement activities in one platform.

3. Problem It Solves

- College operations are often distributed across separate systems and manual processes.
- Students and faculty may need different platforms or communication channels to access academic information.
- Academic, examination, finance, and placement activities require coordinated information sharing.
- Administrators need role-specific dashboards and controlled access to institutional information.

Nivara aims to bring these activities together through a unified, role-based platform.

4. Main Features

The system supports eight user roles:

1. Student: Attendance, assignments, academic results, fees, announcements, and placement opportunities.
2. Faculty: Attendance management, assignments, marks, and student academic activities.
3. HOD: Department-level academic oversight, faculty management, and approvals.

4. Principal / Dean: Institution-level oversight and performance monitoring.
5. ERP Administrator: User management, permissions, and system configuration.
6. Exam Cell: Examination management, schedules, marks, and results.
7. Finance & Accounts: Fee management, payment records, receipts, and financial information.
8. Placement Officer: Company drives, student applications, eligibility, and placement tracking.

Additional capabilities include role-based navigation, dashboard metrics, shared data workflows, authentication, database-backed operations, and responsive layouts.

The exact availability and completion status of individual features should be confirmed against the current build.

5. Screenshots Required

Capture these screens from the current application:

- Login Page: Authentication and entry into the ERP.
- Student Dashboard: Student overview, attendance, assignments, and academic information.
- Faculty Dashboard: Faculty tools for attendance, assignments, and marks.
- HOD Dashboard: Department-level overview and management tools.
- Principal / Dean Dashboard: Institution-level summary and oversight.
- ERP Administrator Dashboard: User and system administration.
- Exam Cell Dashboard: Examination and result management.
- Finance Dashboard: Fees and financial operations.
- Placement Dashboard: Recruitment drives and student applications.
- Cross-role Workflow: A real example of an action in one role appearing in another role's dashboard.

For the PDF, prioritize the login screen, two or three representative dashboards, and one connected workflow. Include the other role screens if space allows.

6. Technology Stack

- Frontend: React 19
- Build Tool: Vite 8
- Language: JavaScript / JSX
- Styling: Vanilla CSS with shared design tokens and layout/component stylesheets
- Database / Backend Service: Supabase
- Authentication and Access Control: Existing authentication and role-based authorization implementation
- Database Security: Supabase Row Level Security (RLS), where configured
- Icons: Lucide React
- Persistence: Supabase with the application's existing data and fallback architecture, subject to current implementation verification

7. Category

Enterprise Software / EdTech / College ERP

Secondary category: Full-Stack Web Development.

8. Project Status

In development — portfolio showcase preparation and final verification.

The application has undergone dashboard redesign, database foundation work, CRUD implementation, and cross-role workflow improvements. The final build status, demo-data isolation, deployment status, and complete workflow verification must be confirmed before describing it as production-ready or live.

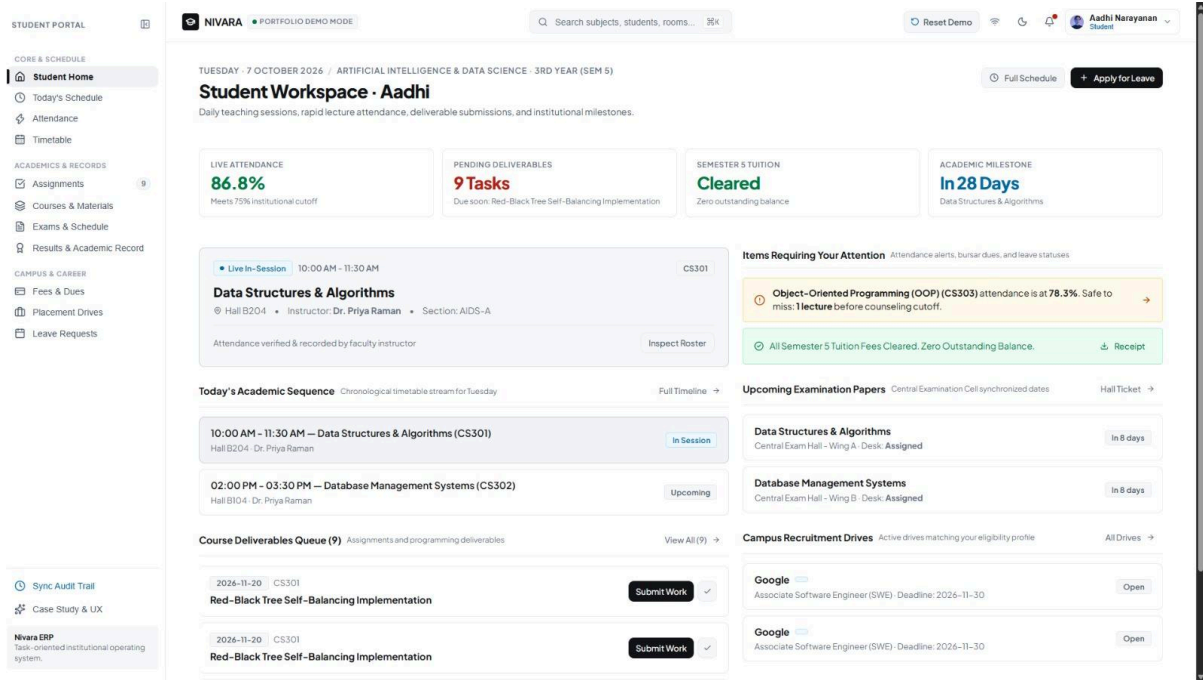
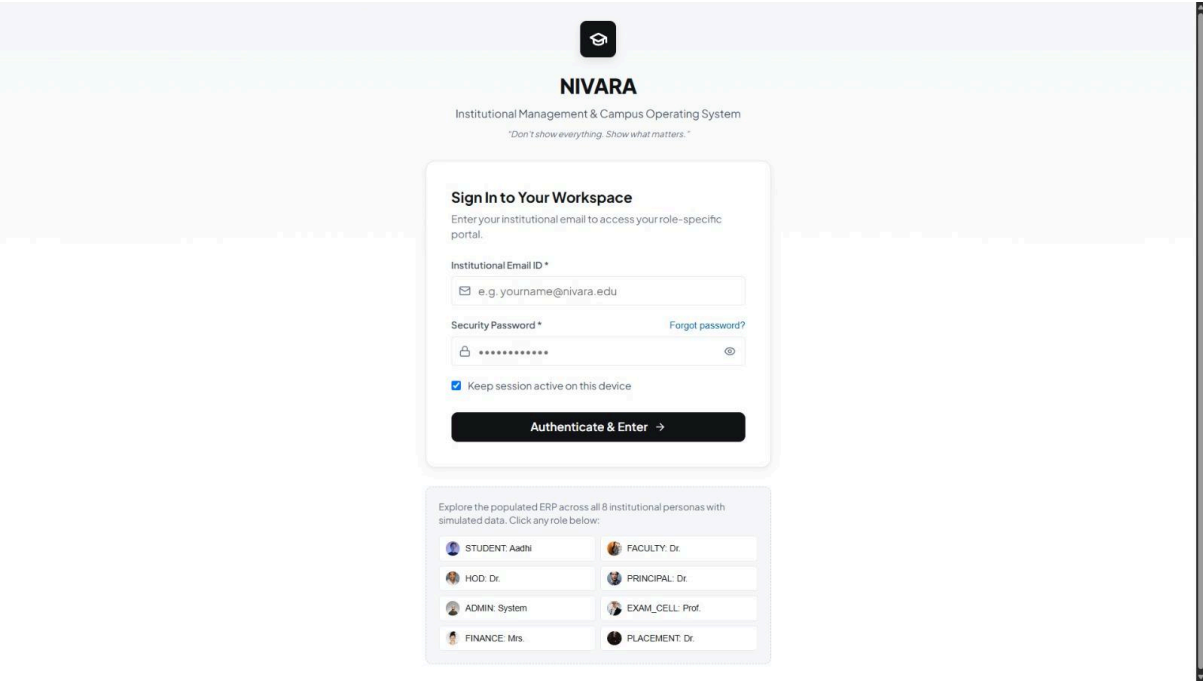
9. Team Contributions

Nivara ERP is a Tetravate team project.

The project work has included:

- Designing and implementing role-specific dashboards for eight user roles.
- Developing the React-based application structure and shared UI components.
- Implementing role-aware navigation and access-control workflows.
- Building academic, examination, finance, and placement management interfaces.
- Integrating Supabase database infrastructure and schema migrations.
- Implementing CRUD operations and cross-role data workflows.
- Improving responsive layouts, dashboard usability, and error handling.
- Preparing a fictional-data demonstration experience for the Tetravate portfolio.

Contribution disclosure: These describe the project areas worked on by the team. Individual ownership and the exact implementation completed by each member should be documented separately rather than attributing all features to one person.



AI/ML

MISTIQ

Project Name

MISTIQ 2.0 — AI-Powered Engineering Intelligence and Failure Prediction Platform

Purpose

To continuously analyze software repositories, identify engineering risks, track project evolution, detect recurring failure patterns, and predict potential future failure categories using historical project observations.

Problem

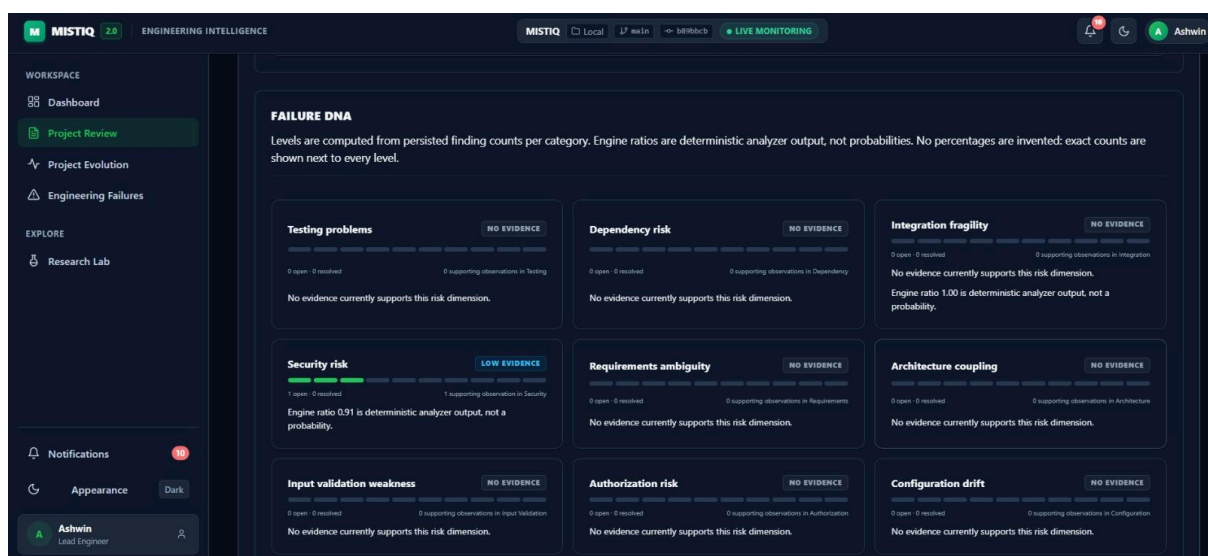
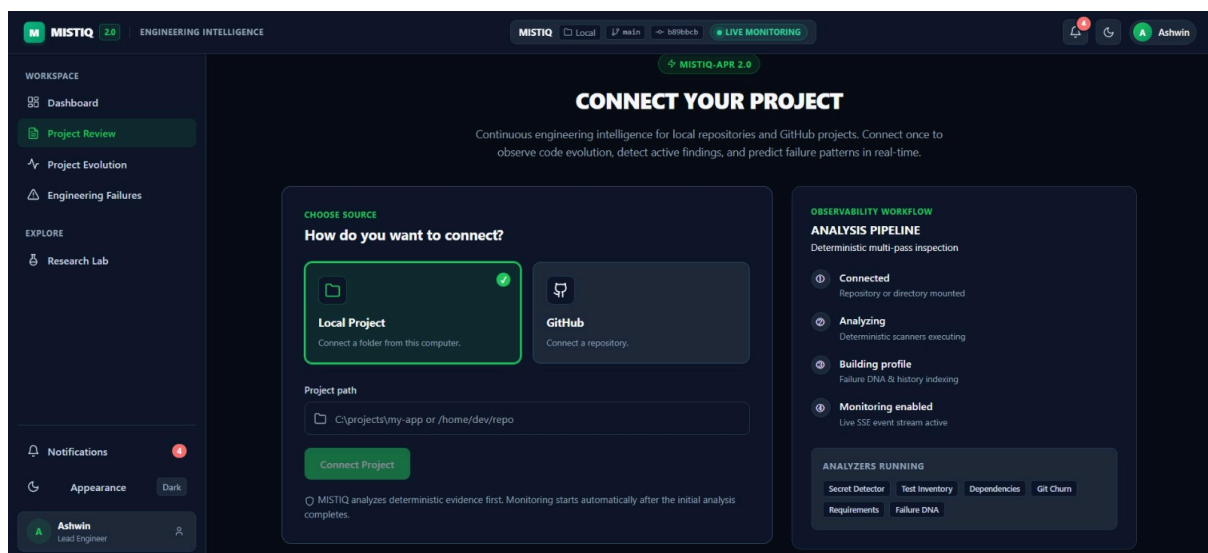
Modern software projects contain interconnected code, APIs, dependencies, configurations, tests, and deployment components. Conventional code analysis tools often identify individual issues but provide limited insight into how engineering problems evolve across reviews, which failures repeatedly occur, and what types of failures may emerge next.

Developers need a unified platform that connects code-level evidence with historical engineering failures to support better prioritization, faster remediation, and more informed software maintenance.

Main Features

1. Automated Project Analysis — Analyze local repositories and public GitHub projects to detect engineering issues using deterministic analyzers.
2. Live Project Monitoring — Detect meaningful file and Git changes, trigger incremental analysis, and update findings automatically.
3. Engineering Risk Detection — Identify security vulnerabilities, hard-coded secrets, missing tests, configuration problems, integration issues, and other engineering risks.
4. Failure History and Lifecycle Tracking — Track new, active, persistent, reopened, and resolved findings across project reviews.

5. Project Evolution Analysis — Compare consecutive reviews to understand changes in findings, severity, recurrence, and resolution.
6. Failure Pattern Recognition — Analyze repeated findings and historical transitions between failure categories.
7. MISTIQ-APR Next-Failure Prediction — Use historical engineering observations and machine learning to estimate which failure categories may appear next, subject to data availability.
8. Project Failure DNA — Build an evidence-based engineering risk profile across dimensions such as security, testing, architecture, configuration, and integration.



SAKSHA

Got it — here are the details for **SAKSHA specifically**, without renaming it to DRISHYAM. I've kept the structure aligned with the requirements for your project portfolio PDF.

SAKSHA — AI Crime Intelligence & Analytics Platform

1. Project Name

SAKSHA – AI Crime Intelligence & Analytics Platform

2. Purpose

SAKSHA is an AI-powered crime intelligence and analytics platform designed to help law enforcement personnel analyze criminal records, identify relationships between entities, and discover potentially significant connections across multiple investigations.

The platform aims to bring investigative data together in a unified system to support criminal network analysis and data-driven investigations.

3. Problem It Solves

Criminal investigation data is often distributed across multiple case records, FIRs, individuals, phone numbers, vehicles, and locations. Analyzing these records independently makes it difficult to identify hidden relationships and recurring patterns.

SAKSHA addresses this problem by organizing investigative data into a connected information system that supports relationship discovery, network visualization, and analytical exploration.

Key challenges addressed:

- * Fragmented criminal investigation records.
- * Difficulty identifying connections between suspects and cases.
- * Limited visibility into complex criminal networks.
- * Time-consuming manual analysis of investigative information.
- * Difficulty identifying geographic and relational patterns across cases.

4. Main Features

A. Criminal Network Analysis

- * Represents people, cases, phone numbers, vehicles, and organizations as connected entities.
- * Helps investigators explore direct and indirect relationships.
- * Supports multi-hop relationship discovery.

B. AI-Assisted Crime Intelligence

- * Supports analytical exploration of criminal investigation data.
- * Helps identify potentially significant relationships and patterns.
- * Provides a foundation for AI-assisted investigative insights.

C. Case and FIR Management

- * Organizes criminal case information.
- * Connects relevant entities with their associated cases.
- * Supports centralized access to investigative records.

D. Entity Relationship Mapping

- * Connects individuals, organizations, vehicles, phone numbers, and cases.
- * Supports exploration of relationships between entities.
- * Helps investigators understand the broader context of an investigation.

E. Geospatial Crime Analytics

- * Supports location-based exploration of investigative information.
- * Helps visualize geographic patterns in available records.
- * Provides a foundation for hotspot and spatial analysis.

F. Timeline Analysis

- * Organizes events chronologically.
- * Supports investigation of relationships between events and cases.

G. Investigation Dashboard

- * Provides a centralized interface for accessing investigative information.
- * Organizes analytical views and case-related data.

Note: These features describe the intended platform capabilities. Confirm which ones were implemented in the version you are presenting.

5. Screenshots and What Each Shows

Include screenshots from the actual SAKSHA application.

Screenshot	What It Shows	
-----	-----	
Main Dashboard	Overview of the crime intelligence platform.	
Criminal Network Graph	Visual connections between people, cases, vehicles, and other entities.	
Case Management	Case records and associated investigative information.	
Entity Profile	Information about an individual or entity and its connections.	

Geospatial Analytics	Map-based visualization of crime-related information.	
Timeline View	Chronological representation of investigative events.	
Analytics Dashboard	Graphs, statistics, and analytical findings, if implemented.	

****Recommended screenshots:**** Main Dashboard, Criminal Network Graph, Entity Profile, Case Management, and Geospatial Analytics.

6. Technology Stack

Based on the architecture discussed during the SAKSHA development phase:

Component	Technology	
-----	-----	
Frontend	React	
Backend	Python with FastAPI	
Primary Database	PostgreSQL / Supabase	
Graph Database	Neo4j — part of the earlier architecture	
API Architecture	REST APIs	
Analytics	Graph-based relationship and geospatial analysis	
Development Tools	Git and associated development tooling, if used	

****Important:**** The original architecture included PostgreSQL and Neo4j. Later, we discussed rebuilding the platform around Supabase and SQL. List the technologies used in the actual SAKSHA version rather than combining technologies from different architectural versions.

7. Project Category

****Primary category:**** Artificial Intelligence & Crime Analytics

****Secondary categories:****

- * Criminal Intelligence Systems
- * Graph Analytics
- * Geospatial Analytics
- * Law Enforcement Technology
- * Data Analytics
- * Decision Support Systems

8. Project Status

****Status:** Academic / hackathon project.

SAKSHA was developed as an AI Crime Intelligence & Analytics Platform associated with the SIH26189 problem statement.

The project explored the integration of investigative data, entity relationships, and analytical capabilities into a unified platform.

The current deployment status and the implementation completeness of individual features need confirmation before describing it as a live or production-ready system.

9. Contributions / Implementation

Based on the project architecture and development direction discussed previously, the implementation areas included:

- * Designed the architecture for a crime intelligence and analytics platform.
- * Structured investigative data around cases and interconnected entities.
- * Planned APIs for communication between the frontend and backend.
- * Designed graph-based relationship exploration for criminal network analysis.
- * Worked on the database architecture involving PostgreSQL and an earlier Neo4j-based approach.
- * Planned the migration toward a Supabase-based SQL architecture.
- * Defined geospatial analytics and investigation-oriented data exploration as core capabilities.

****Contribution note:**** These points describe the documented project direction. For your final PDF, include only the work you and your teammates actually completed, distinguishing implemented features from planned functionality.

10. Portfolio Summary

Use this concise summary at the beginning of the SAKSHA section.

****SAKSHA — AI Crime Intelligence & Analytics Platform****

SAKSHA is an AI-assisted crime intelligence platform designed to help investigators analyze criminal records, discover relationships between entities, and explore connected criminal networks. The system focuses on integrating case information, entity relationship mapping, and geospatial analytics to support data-driven investigations. Its architecture explored a React frontend, FastAPI backend, PostgreSQL, and Neo4j for structured data management and graph-based analysis.

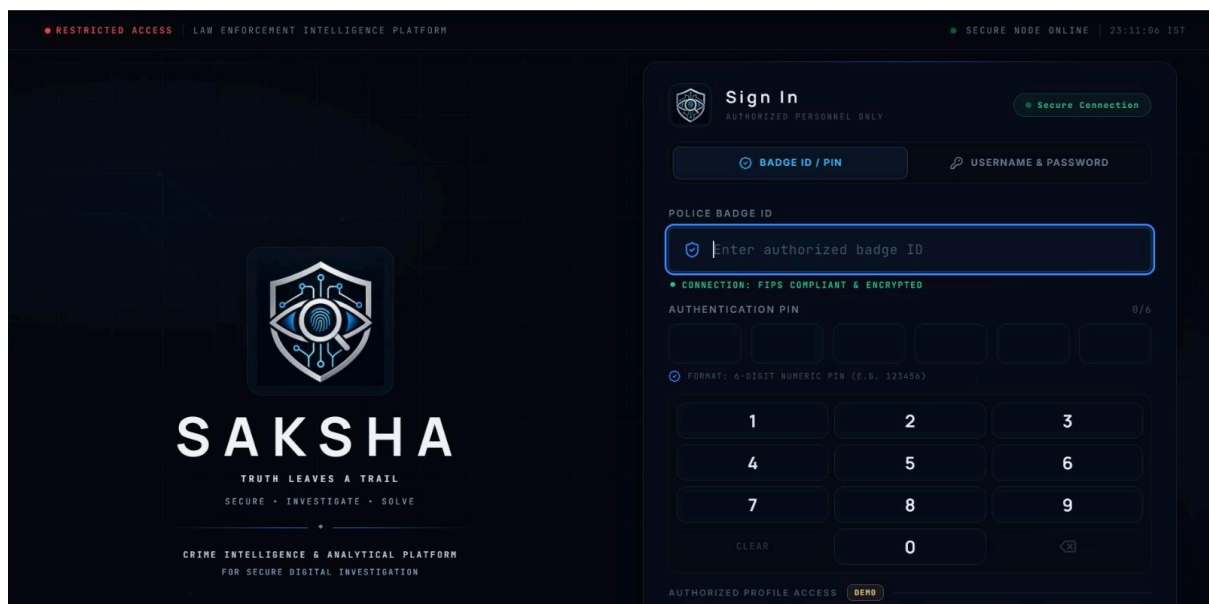
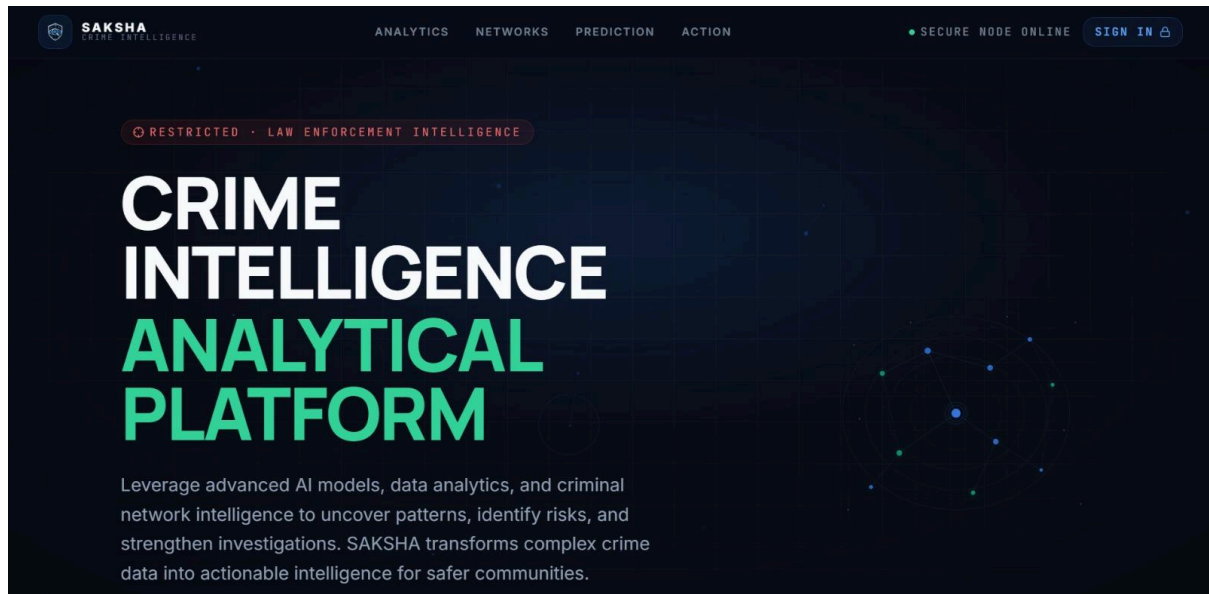
11. What Makes SAKSHA Stand Out?

1. ****Relationship Discovery:**** Connects investigative entities to help uncover direct and indirect relationships across cases.
2. ****Graph-Based Intelligence:**** Uses interconnected data to explore potentially significant criminal networks.
3. ****Geospatial Analysis:**** Adds a geographic perspective to investigative data.
4. ****Integrated Investigation Workflow:**** Aims to bring case records, entities, and analytical insights together in one platform.

Final recommendation for your PDF

Use the title ****SAKSHA — AI Crime Intelligence & Analytics Platform**** and emphasize its graph-based criminal network analysis and investigative intelligence capabilities.

Before finalizing the PDF, verify the exact features implemented in SAKSHA and use screenshots from that version. This will keep the portfolio technically accurate and distinguish SAKSHA from its later iteration, DRISHYAM.



MOBILE APPLICATION

AGA MASALA

AGS MASALAS — Project Details

1. Project Name

****AGS MASALAS – Inventory & Point-of-Sale (POS) Billing Application****

2. Purpose

A mobile-first billing and inventory management application developed for AGS Masalas to simplify daily retail operations. The application manages product stock, generates customer bills, tracks sales, maintains customer details, and handles pending payments.

3. Problem It Solves

Small retail businesses often manage inventory, billing, customer records, and pending balances manually. This can lead to calculation errors, difficulty tracking stock, time-consuming bill preparation, and challenges maintaining sales records.

AGS MASALAS digitizes these operations through a centralized application with billing automation, inventory tracking, and customer management.

4. Main Features

- * ****Product & Inventory Management:**** Maintain masala product details and monitor available stock.
- * ****Billing System:**** Generate bills with automatic quantity × price calculations.
- * ****Automatic Stock Updates:**** Deduct sold quantities from inventory.
- * ****Customer Management:**** Maintain customer details and reuse existing customer information.
- * ****Pending Balance Tracking:**** Record outstanding customer payments.
- * ****Invoice Generation:**** Generate, download, and print customer invoices.
- * ****WhatsApp Invoice Sharing:**** Share invoice information through WhatsApp.
- * ****Sales History:**** Maintain records of previous transactions.
- * ****Sales Filtering:**** Review sales using date-based filters, including weekly and monthly views where implemented.
- * ****Offline Support:**** Cache application data locally so essential operations can continue during network interruptions.
- * ****Cloud Synchronization:**** Synchronize application data with Supabase.
- * ****Mobile-Friendly Interface:**** Provide an interface designed for convenient use during everyday retail operations.

5. Screenshots and What Each Shows

Include screenshots of the following screens in your PDF. Use actual screenshots from the application rather than mockups.

Screenshot	What It Demonstrates
Dashboard	Overview of billing, inventory, and key business operations.
Product Management	Product listings, prices, and stock quantities.
Add/Edit Product	Interface for entering or modifying product information.
Billing Screen	Product selection, quantity entry, price calculation, and bill preparation.
Customer Management	Customer information and selection of existing customers.
Invoice Screen	Generated bill with customer and purchase details.
Sales History	Previously completed transactions and sales records.
Pending Payments	Customer balances and outstanding payments.
Inventory/Stock View	Available stock and changes following sales.
Cloud/Sync Status	Synchronization or offline-state indicators, if available in the current interface.

****Recommended screenshots for a compact portfolio:**** Dashboard, Billing Screen, Invoice Screen, Inventory Screen, and Sales History.

6. Technology Stack

Based on the latest implementation details available for this project:

Component	Technology
Frontend	React 19
Build Tool	Vite 8
Programming Language	TypeScript 5.9
Styling	Tailwind CSS v4
UI Animations	Framer Motion
State Management	Zustand
Local Persistence	Zustand Persist
Offline Data Storage	Dexie.js with IndexedDB
Backend / Cloud Database	Supabase
Mobile Application	Capacitor-based Android application
Deployment Platforms	Android and web browser

****Important:**** An earlier version of AGS MASALAS was planned as a Flutter/Dart application using SQLite. The technology stack above describes the more recent React-based implementation. Use the stack corresponding to the actual version you are showcasing.

7. Project Category

****Primary category:**** Business Software / Point-of-Sale (POS)

****Secondary categories:****

- * Inventory Management
- * Retail Management
- * Business Process Automation
- * Mobile Application Development

8. Project Status

****Status:** Functional application with Android and browser support.

The application includes billing, inventory management, customer records, invoice generation, and cloud synchronization functionality.

The Android application uses a Capacitor wrapper, while the browser version provides web access.

****Note:**** Confirm the current deployment status before describing it as publicly deployed or production-ready. A functional local or installed application is not necessarily a publicly accessible live application.

9. Contributions / Implementation

The following describes the application's implementation. Include only the work you personally completed or your team actually implemented.

- * Developed a digital POS billing workflow for masala retail operations.
- * Implemented product and inventory management.
- * Built automated bill calculations based on product quantities and prices.
- * Integrated customer records and pending payment tracking.
- * Implemented invoice generation with printing, downloading, and WhatsApp-sharing options.
- * Developed sales history and transaction tracking.
- * Integrated local data persistence to support offline usage.
- * Connected the application to Supabase for cloud data synchronization.
- * Implemented a responsive interface for mobile and browser usage.
- * Packaged the web application for Android using Capacitor.

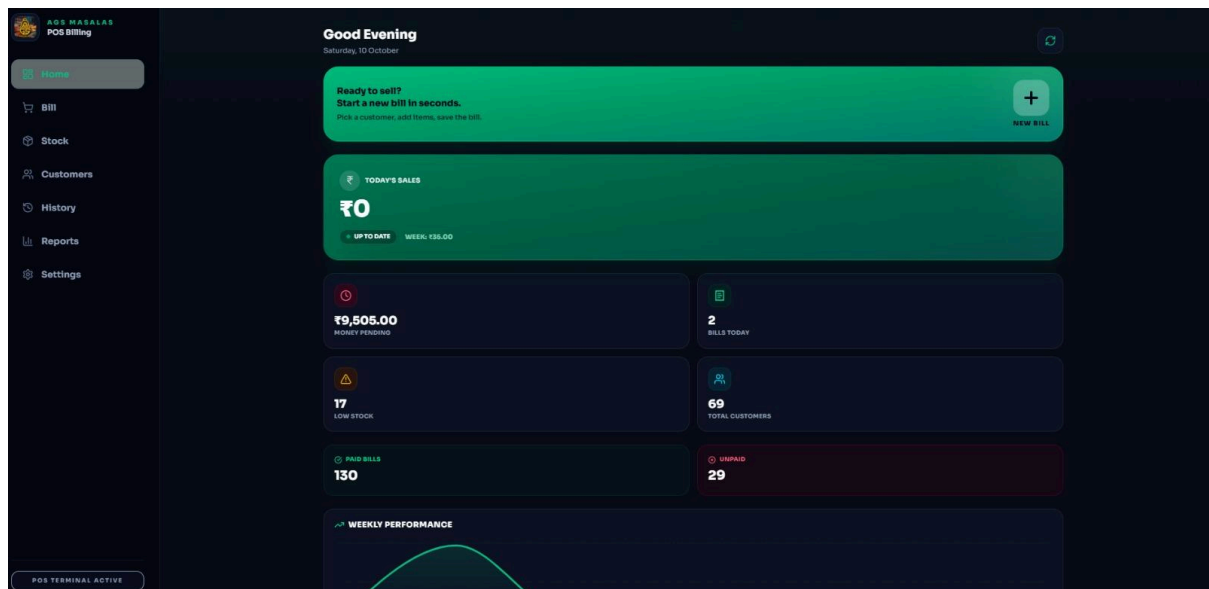
If this was an individual project, describe these as your contributions. If it was a team project, identify which features you personally developed rather than claiming the entire implementation.

10. Suggested Project Summary for Your Portfolio

Use this short summary at the beginning of the AGS MASALAS section.

AGS MASALAS — Retail POS & Inventory Management System

AGS MASALAS is a mobile-first retail management application designed to digitize billing, inventory tracking, customer management, and sales operations for a masala business. It features automated bill calculations, stock management, invoice generation, pending payment tracking, offline data persistence, and cloud synchronization. Built using React, TypeScript, Tailwind CSS, and Supabase, the application supports both browser-based access and Android usage through Capacitor.





AGS MASALAS

Simple billing for your shop

POS

Good Evening

Saturday, 10 October



Ready to sell?
Start a new bill in seconds.

Pick a customer, add items, save the bill.



NEW BILL



TODAY'S SALES

₹0

● UP TO DATE

WEEK: ₹35.00



₹9,505.00

MONEY PENDING



2

BILLS TODAY



17



69



Home



Bill



Stock



Customer



History



Reports



Settings

Pustakabhrita

1. Project Name

Pustakabhrita – Tuition Fee Management System

2. Purpose

An offline-first application designed to simplify tuition centre administration by managing student records, monthly fees, payment history, outstanding balances, and fee reminders.

3. Problem It Solves

Tuition teachers often manage student information and monthly fee payments manually using notebooks or spreadsheets. This makes it difficult to track outstanding fees, maintain payment histories, and remind parents about overdue payments.

Pustakabhrita digitizes these operations through a centralized application that stores student records, tracks fee payments, and helps manage pending balances.

4. Main Features

Student Management: Add, view, update, and delete student records.

Fee Management: Manage monthly tuition fees for individual students.

Payment Tracking: Record payments and maintain payment status.

Pending Fee Tracking: Identify students with outstanding balances.

Payment History: Review previous fee transactions.

WhatsApp Fee Reminders: Open WhatsApp with a prefilled fee reminder message for the parent or student.

Student Contact Management: Maintain separate student and parent/guardian phone numbers.

Offline Functionality: Store and access records locally without requiring a continuous internet connection.

Data Management: Back up, export, and import application data.

Automatic Data Cleanup: Remove older fee records according to the three-calendar-month retention policy while preserving current outstanding balances.

5. Screenshots and What Each Shows

Include actual screenshots of these screens in your portfolio.

Screenshot	What It Shows
Dashboard	Overview of student records and fee-management activities.
Student List	Registered students and their details.
Add Student	Form for entering student information and contact details.
Student Profile	Individual student information, fee amount, and payment status.
Fee Management	Monthly fee records and payment tracking.
Payment History	Previously recorded payments.
Pending Fees	Outstanding balances that require collection.
WhatsApp Reminder	Fee reminder message prepared for a parent or student.
Settings / Data Management	Application preferences and backup or import/export options.

Recommended: Use five to seven screenshots showing the dashboard, student list, student profile, fee tracking, payment history, and WhatsApp reminder workflow.

6. Technology Stack

Component	Technology
-----------	------------

Frontend React
Programming Language TypeScript
Build Tool Vite
Android Integration Capacitor
Local Database SQLite
Architecture Offline-first application
Messaging Integration WhatsApp deep links

This reflects the technology direction established for Pustakabhrita. Confirm the final implementation before presenting every component as completed.

7. Project Category

Primary category: Business Management Software

Secondary categories:

Education Technology (EdTech)
Fee Management Systems
Administrative Automation
Mobile Application Development

8. Project Status

Status: Application project / prototype — confirm current implementation and deployment status.

The application was designed around offline fee management and local data storage. Whether the latest version is fully functional, installed on Android, or publicly deployed needs to be established before claiming a specific release status.

9. Contributions / Implementation

Based on the project specifications we previously discussed, the intended implementation includes:

Designed the student and tuition fee management workflow.
Defined student record management and monthly fee tracking.
Planned payment recording, outstanding balance calculations, and payment history.
Integrated WhatsApp deep links for preparing fee reminder messages.
Designed separate contact fields for student and parent/guardian phone numbers.
Planned local database storage to support offline operation.
Specified automatic cleanup of older fee records while retaining outstanding balances.
Planned data backup, import, and export functionality.
Selected React, TypeScript, Vite, Capacitor, and SQLite for the application.

Important: These are the project requirements and implementation direction we discussed. For your final PDF, distinguish features you actually implemented from features that remain planned.

10. Portfolio Summary

PUSTAKABHRITA — Offline Tuition Fee Management Application

Pustakabhrita is an offline-first tuition fee management application designed to simplify student administration, monthly fee tracking, payment history, and outstanding balance management. It includes WhatsApp-based fee reminders, separate student and parent contact details, local data storage, and automated record-retention rules. The application was designed using React, TypeScript, Vite, Capacitor, and SQLite.

Dashboard

Pending this month

₹7,400

₹8,000 expected • 9 student(s)



[View Pending](#)

[View Overdue](#)

Total Students

10

10 active



Expected in Oc...

₹8,000



Collected

₹600



Pending

₹7,400



Overdue

₹0



Collection Rate

7.5%

1 paid • 0 partial ...



Quick Actions



[Add Student](#)



[Generate
Monthly Fees](#)



Dashboard



Students



Fees



History



Settings

PROJECT MANAGEMENT

AGA-AI-ASSISTANT

Project: AGA AI Assistant

1. Project Name:

AGA AI Assistant

2. Purpose:

To develop an AI-powered assistant that understands user requests and helps complete tasks through intelligent responses, contextual understanding, and AI-driven assistance.

3. Problem It Solves:

Traditional assistants often struggle with complex requests, contextual follow-ups, and multi-step tasks. AGA AI Assistant aims to make human–AI interaction more intelligent, efficient, and user-friendly.

4. Main Features:

- Natural language interaction with users.
- AI-powered question answering and response generation.
- Context-aware conversations.
- Assistance with user queries and tasks.
- Extensible architecture for integrating additional AI capabilities.

5. Screenshots:

- **Main Interface:** Shows the assistant's primary user interface.
- **Chat Interface:** Demonstrates user prompts and AI-generated responses.
- **Task Execution:** Shows an example of the assistant processing a user request, if implemented.
- **Additional Features:** Screenshots of other implemented modules or settings.

6. Technology Stack:

- **Programming Language:** To be confirmed.

- **AI/LLM:** To be confirmed.
- **Frameworks and Libraries:** To be confirmed.
- **Database and Storage:** To be confirmed.
- **Tools and APIs:** To be confirmed based on implementation.

7. Category:

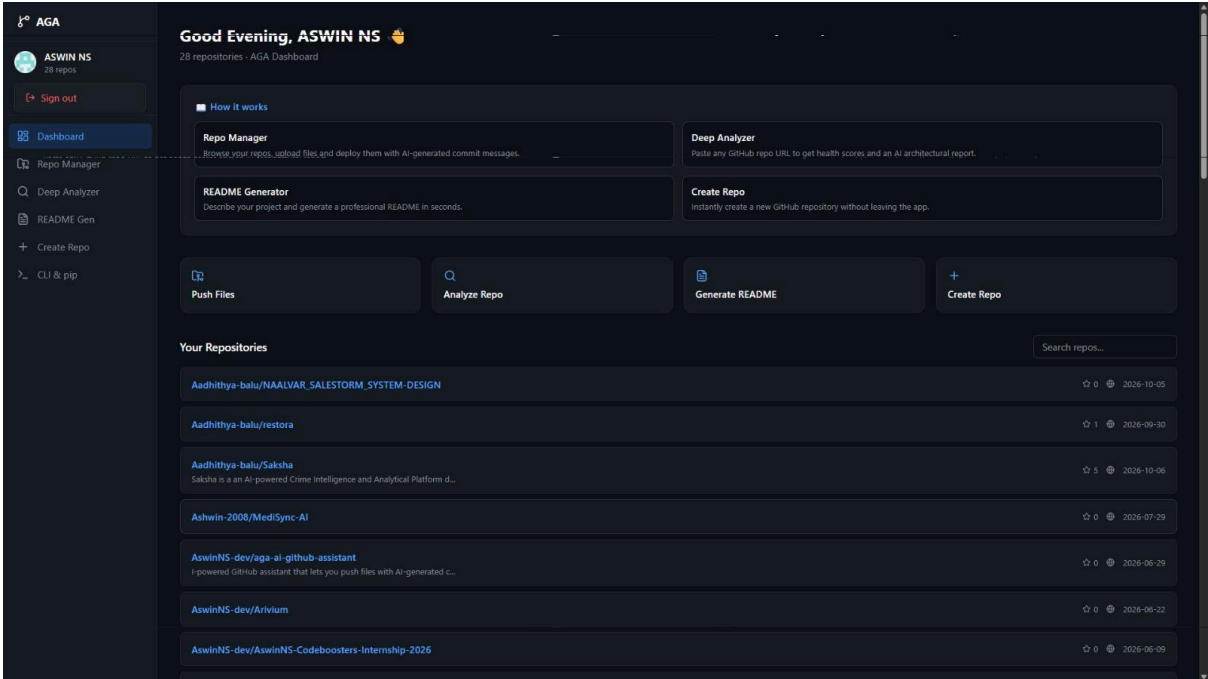
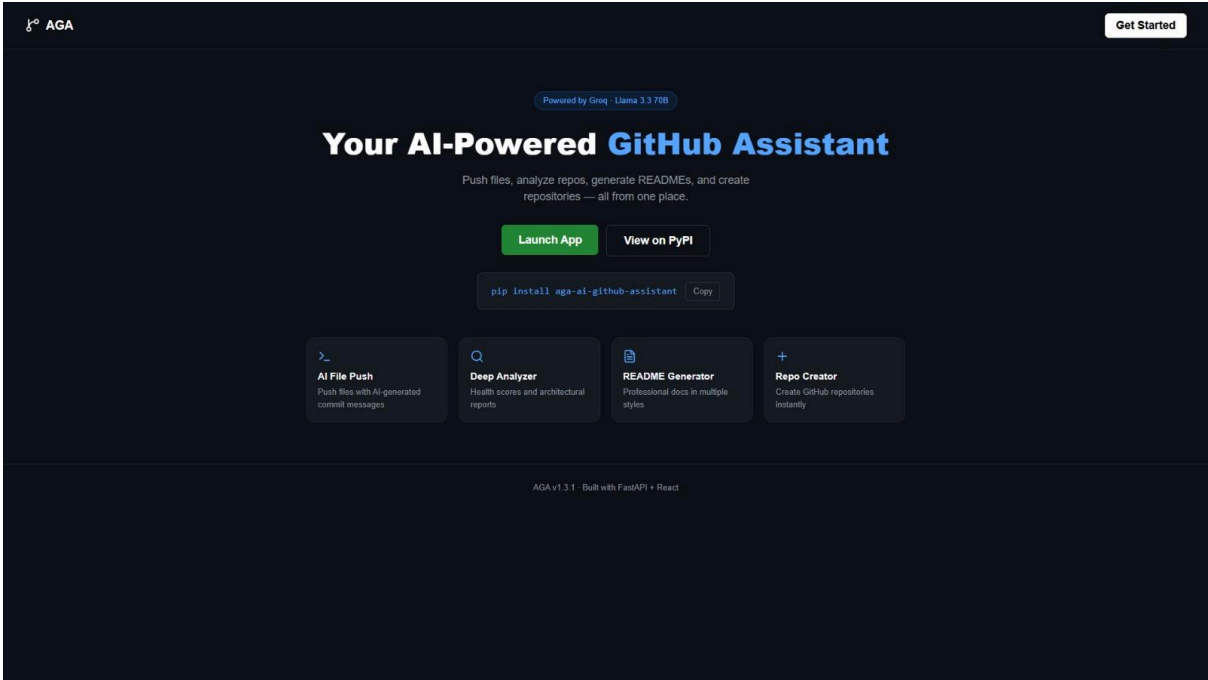
Artificial Intelligence / Generative AI / AI Assistant.

8. Project Status:

To be confirmed — prototype, deployed demo, or live application.

9. Contributions:

Document the actual individual and team contributions, including AI integration, prompt processing, backend development, frontend development, conversation management, tool integration, and testing, wherever implemented.



TETRAVATE WORKSPACE

Project Details — Tetravate Workspace

1. Project Name

Tetravate Workspace — Collaborative Software Development Platform

2. Purpose

Tetravate Workspace is a shared development environment created by the Tetravate team to organize software projects, collaborate through GitHub, manage code changes, and coordinate development among team members.

3. Problem It Solves

When multiple developers work on different parts of a project, code can become disorganized, changes can conflict, and collaboration can be difficult to manage. Tetravate Workspace provides a centralized repository and a structured Git workflow to help the team collaborate efficiently and maintain organized project code.

4. Main Features

Shared GitHub repository for team collaboration.

Branch-based development for individual features and fixes.

Git commits and version history to track code changes.

Pull requests for reviewing and merging contributions.

Centralized organization of frontend and backend code.

Team collaboration with repository access permissions.

Support for developing, testing, and deploying web applications.

Integration with deployment platforms such as Vercel for applicable projects.

5. Screenshots and What Each Shows

GitHub Repository: Shows the central repository, project files, and code structure.

Branches and Commits: Shows how team members manage separate branches and track changes.

Pull Requests: Shows the process of reviewing and merging code contributions.

Project Directory: Shows the organization of frontend, backend, and configuration files.

Live Deployment: Shows a web application developed by the team and deployed online, if applicable.

6. Technology Stack

Version Control: Git

Code Hosting and Collaboration: GitHub

Frontend: React, JavaScript/TypeScript, HTML, CSS, and Tailwind CSS, depending on the project

Backend: Node.js and related tools, depending on the project

Database: Supabase or PostgreSQL, where used

Deployment: Vercel, where applicable

Development Tools: Visual Studio Code and npm

7. Category

Software Development · Team Collaboration · Web Application Development

8. Status

Active development workspace. Individual applications maintained in the workspace may have different statuses, such as prototype, academic project, or deployed demo.

9. Contributions

The Tetravate team established and organized the shared GitHub workspace, configured repository access for team members, and adopted a branch-based development workflow. The team uses Git for version control, commits for tracking changes, and pull requests for reviewing contributions. The workspace supports collaborative development of web applications and other software projects.

